DuocUC® INFORMÁTICA Y TELECOMUNICACIONES

SPRING BOOT: BUENAS PRACTICAS

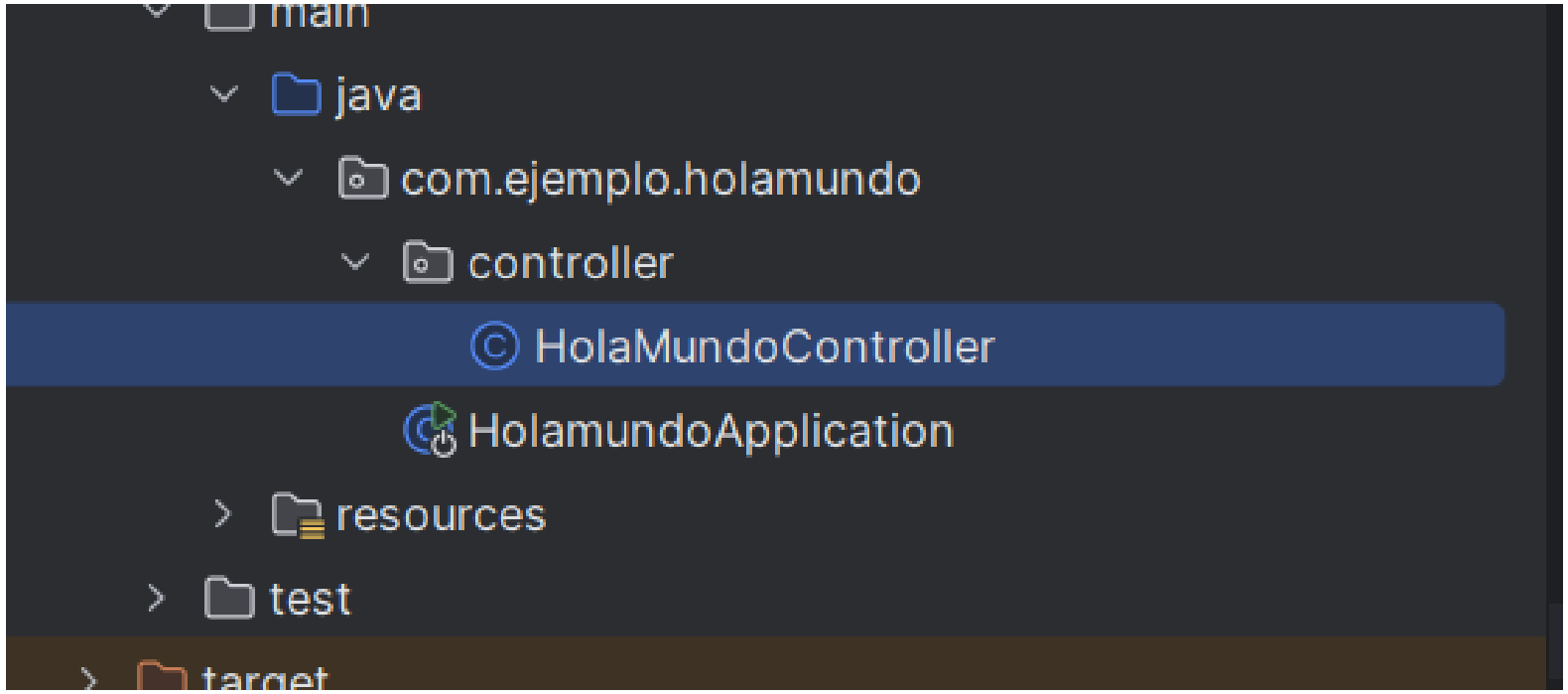
DESARROLLO FULLSTACK I

DSY1103

FORMAMOS PERONAS PARA UNA SOCIEDAD MEJOR

En el capítulo anterior…

¿Qué hace el package **controller**?



ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

DuocUC®

Sabías que el formato **JSON (JavaScript Object Notation)** es uno de los más populares para el intercambio de datos en las **APIs REST.**

```
{
    "vehiculo_id": 100,
    "patente": "ABCD29",
    "anio": 2013,
    "persona_id": 20003
}
```

• Contenidos

- Arreglos y Colecciones.
- Validaciones y manejo de errores.
- Buenas practicas en microservicios.

01

Arreglos y Colecciones

¿Qué son las colecciones y arreglos en Java / Spring Boot?

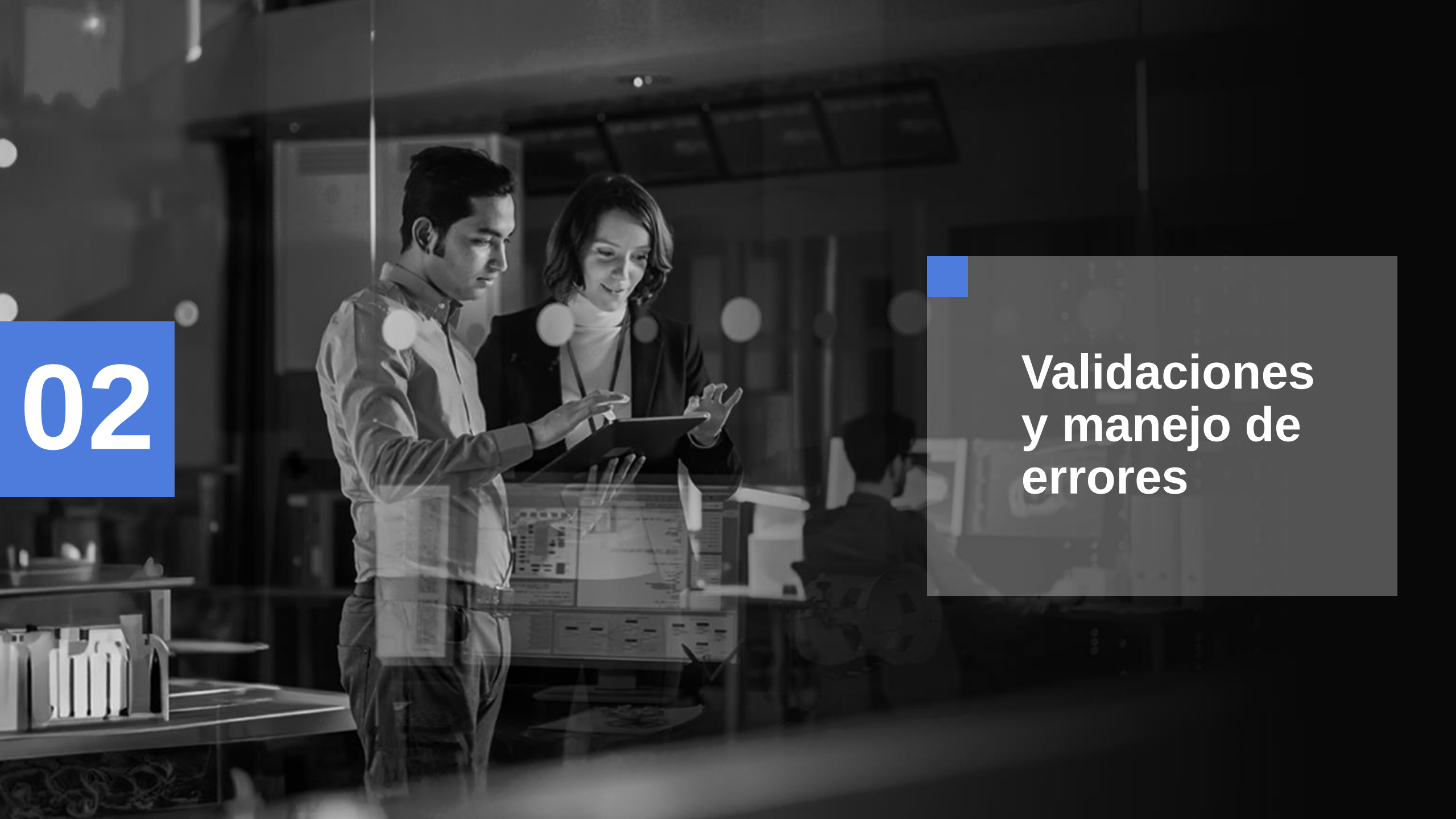
Concepto	Descripción	Ejemplo simple
Arreglo (`Array`)	Estructura de tamaño fijo. No puede crecer ni reducirse.	`int[] numeros = {1,2,3,4};`
Lista (`List`)	Colección dinámica que puede cambiar de tamaño.	`List<String> productos = new ArrayList<>();`
Mapa (`Map`)	Estructura que guarda pares *clave–valor*.	`Map<String, Integer> stock = new HashMap<>();`
Set (`HashSet`)	Conjunto sin elementos duplicados.	`Set<String> nombres = new HashSet<>();`

En Spring Boot, las colecciones se usan constantemente dentro de servicios para manejar listas de objetos que luego se devuelven como JSON.

IR A JAVA COLLECTIONS FRAMEWORK

```java
@RestController
@RequestMapping("/api/v1/productos")
public class ProductoController {

    private final List<String> productos = new ArrayList<>(List.of("Cuaderno", "Lápiz", "Regla"));

    @GetMapping
    public List<String> listar() {
        return productos;  // → JSON
    }

    @PostMapping
    public String agregar(@RequestParam String nombre) {
        if (nombre.isBlank()) return "❌ El nombre no puede estar vacío";
        productos.add(nombre);
        return "✅ Producto agregado";
    }
}
```

IR A SPRING BOOT FRAMEWORK



02

Validaciones y manejo de errores

¿Qué es la validación en Spring Boot?

Spring Boot integra Jakarta Bean Validation (usualmente con Hibernate Validator) para validar automáticamente los datos que llegan a tus endpoints. Se activa al poner el starter de validación en el classpath y usar anotaciones en tus DTOs/modelos

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

IR A JAKARTA

IR A JAVA BEANS VALIDATION

IR A VALIDATION

Anotaciones esenciales

- @NotNull: el campo no puede ser null (pero puede ser vacío si es String).
- @NotBlank: solo para String; no puede ser null, ni vacío, ni solo espacios.
- @Valid: activa la validación de un objeto anidado (por ejemplo, un @RequestBody). Por sí sola no es una restricción, dispara la validación de las que sí lo son.

```java
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;

public class ClienteRequest {

  @NotBlank(message = "El nombre es obligatorio")
  private String nombre;

  @NotBlank(message = "El email es obligatorio")
  private String email;

  @NotNull(message = "La edad no puede ser nula")
  private Integer edad;

  public String getNombre() { return nombre; }
  public String getEmail()  { return email; }
  public Integer getEdad()  { return edad; }
}
```

Controller usando @Valid y devolviendo ResponseEntity

- @Valid sobre @RequestBody hace que se evalúen @NotNull/@NotBlank del DTO automáticamente.
- ResponseEntity permite controlar código HTTP, headers y body de la respuesta

```java
import cl.duoc.demo.dto.ClienteRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/v1/clientes")
public class ClienteController {

  @PostMapping
  public ResponseEntity<String> crear(@Valid @RequestBody ClienteRequest request) {
    // Si el request es inválido, Spring lanzará una excepción de validación ANTES de entrar aquí
    return ResponseEntity.ok("✅ Cliente recibido: " + request.getNombre());
  }
}
```

Manejo de errores con try/catch + ResponseEntity (local)

Si prefieres capturar y devolver mensajes controlados en un método puntual

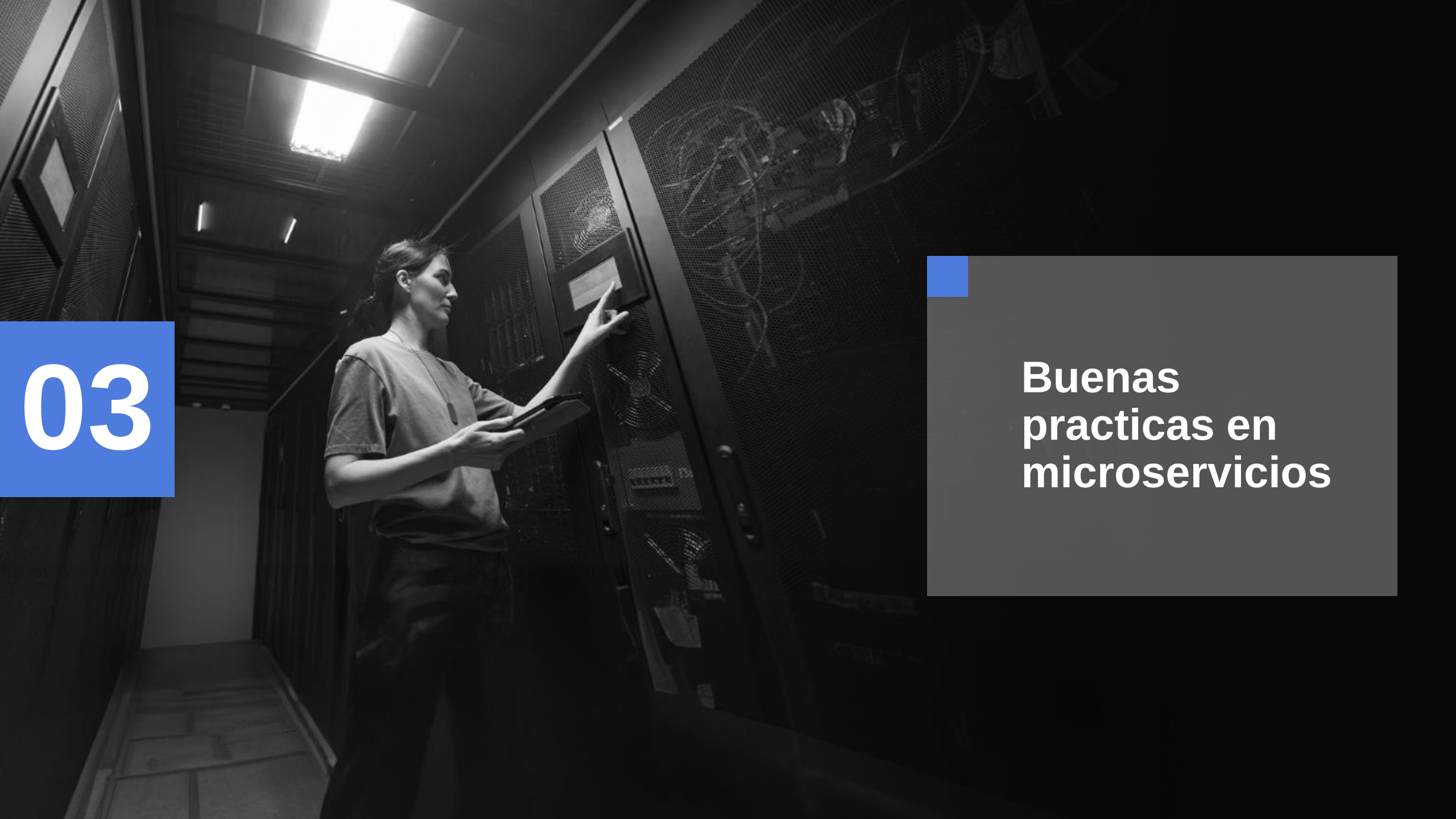
```java
@PostMapping("/pago")
public ResponseEntity<?> pagar(@Valid @RequestBody PagoRequest req) {
  try {
    // lógica de servicio que podría fallar (ej.: saldo insuficiente)
    servicio.procesarPago(req);
    return ResponseEntity.ok().body("✅ Pago procesado");
  } catch (SaldoInsuficienteException e) {
    // 400: error de negocio del cliente
    return ResponseEntity.badRequest().body("❌ " + e.getMessage());
  } catch (Exception e) {
    // 500: error inesperado del servidor
    return ResponseEntity.status(500).body("⚠️ Error interno");
  }
}
```

Manejo de errores de forma global (recomendado)

Para no escribir try/catch en cada método, centraliza con @RestControllerAdvice + @ExceptionHandler

```java
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.bind.MethodArgumentNotValidException;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class ApiExceptionHandler {

  // Errores de Bean Validation (@Valid) en @RequestBody
  @ExceptionHandler(MethodArgumentNotValidException.class)
  public ResponseEntity<Map<String, String>> handleValidation(MethodArgumentNotValidException ex)
    Map<String, String> errores = new HashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(err ->
        errores.put(err.getField(), err.getDefaultMessage())
    );
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errores);
  }
```

```java
  // Otros errores de negocio
  @ExceptionHandler(SaldoInsuficienteException.class)
  public ResponseEntity<Map<String, String>> handleSaldo(SaldoInsuficienteException ex) {
    Map<String, String> error = Map.of("error", ex.getMessage());
    return ResponseEntity.badRequest().body(error);
  }

  // Fallback genérico
  @ExceptionHandler(Exception.class)
  public ResponseEntity<Map<String, String>> handleGeneral(Exception ex) {
    Map<String, String> error = Map.of("error", "Error interno");
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
  }
}
```



03

Buenas practicas en microservicios

• Patrón CSR (Controller – Service – Repository)

Capa	Responsabilidad	Ejemplo
Controller	Recibir y responder solicitudes HTTP.	`@GetMapping("/clientes")`
Service	Contener la lógica de negocio (validaciones, cálculos).	`ClienteService.validarCliente()`
Repository	Acceso a datos (simulado o persistente).	`List<Cliente>` o BD en próximas unidades.

IR A SPRING CORE

• Ventajas de aplicar CSR

- Código más ordenado y reutilizable.
- Facilita pruebas unitarias.
- Permite escalar el proyecto sin romper otras partes.
- Cumple principios SOLID (Single Responsibility).

• Buenas Prácticas al Aplicar CSR en Microservicios

- **Nombrar paquetes por capa**
- **Evitar lógica de negocio en el Controller**

  Solo recibe solicitudes y devuelve respuestas.
- **Mantener métodos cortos y claros**

  Cada método debe realizar una única acción.
- **Usar validaciones consistentes**

  En la capa de servicio (validaciones lógicas simples)

  y en la capa modelo (con anotaciones @Valid, @NotBlank).
- **Retornar respuestas con sentido**

  HTTP200 OK, 201 CREATED, 400 BAD REQUEST, 404 NOT FOUND.
- **Versionar tus endpoints**

  /api/v1/... → permite mantener compatibilidad en el tiempo.

Reflexionemos

- ¿Cuál es la diferencia entre @Controller y @RestController en Spring Boot?
- Explica la estructura de carpetas en un proyecto típico de Spring Boot.
- Describe el flujo de trabajo típico en una aplicación Spring Boot que sigue el patrón CSR (Controller-Service-Repository).
- ¿Cuáles son las dependencias?
- ¿Cómo se añaden manualmente las dependencias en un proyecto Spring Boot utilizando Maven?

DuocUC®

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS ACREDITADO

Comisión Nacional de Acreditación CNA-Chile

DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.